Data Models Query Language

Lecture 1: Introduction & The Relational Model

Week 1

Our Philosophy: From User to Architect

This course is built on one core belief

To truly master data systems, you can't just be a user of the tools. You must understand the fundamental principles that make them work.

We Will Go Deep

- Why does one query run in milliseconds while another takes hours?
- How can thousands of users access the same data without corrupting it?
- What happens if the power fails mid-transaction?

The Goal You will learn to think like a database system designer: making critical trade-offs between consistency, performance, and complexity.

Our Journey: From Theory to Modern Practice

Part 1: The Relational Foundation (Weeks 1-8)

- The Language: **Relational Algebra & SQL**
- Good Design: **E/R Modeling & Normalization**
- Making it Fast: **Storage & Indexing (B+Trees)**

Part 2: Scale & Reliability (Weeks 9-15)

- Handling Chaos: **Transactions (ACID), Concurrency, Recovery**
- Beyond Tables: **NoSQL, Big Data (MapReduce), Cloud DBs**

The Golden Rule: No Object-Relational Mappers (ORMs)

Why?

To become an expert, you must learn to speak the database's native language: **SQL**.

ORM (Abstracted Code)

```
# What the ORM user writes
user = session.get(User, 101)
```

What You WILL Do (Raw SQL)

```
-- What you will write
SELECT id, name, email
FROM users
WHERE id = 101;
```

ORMs hide the complexity—and the power—of the database. In this course, you will learn to **understand and control** that power directly.

What You'll Learn To Optimize

The Problem: A Naive Query

```
SELECT *
FROM billion_row_table
WHERE user_id = 123;
```

Execution Plan: Full Table Scan The database reads every single row. Could take **hours**.

The Solution: An Index

```
CREATE INDEX ON
billion_row_table(user_id);
```

Execution Plan: Index Seek The database navigates a B+Tree structure in a few steps. Takes **milliseconds**.

Integrity in the Age of AI

Encouraged: Responsible Use

- Brainstorming boilerplate code.
- Debugging assistance.
- Improving documentation clarity.

Violation: Generating Core Logic

- Using AI to write core project logic (e.g., complex SQL, ETL code).
- Using AI to generate written analysis or reports.

Transparency is key: Any AI use *must* be documented in your project's `README.md`.

The Data Point of Now

Every second, the modern world generates terabytes of data.

- Financial trades are executed in **microseconds**.
- Social media platforms process millions of **events per second**.
- Recommendation engines make **billions of predictions** daily.

This isn't magic. This is engineering.
This course is about the systems that make this possible—the engines of the data economy.

What is a DBMS? The "Librarian"

- **Database:** A large, integrated collection of data that models a real-world enterprise.
- **DBMS:** Software designed to store and manage databases.

The DBMS provides:

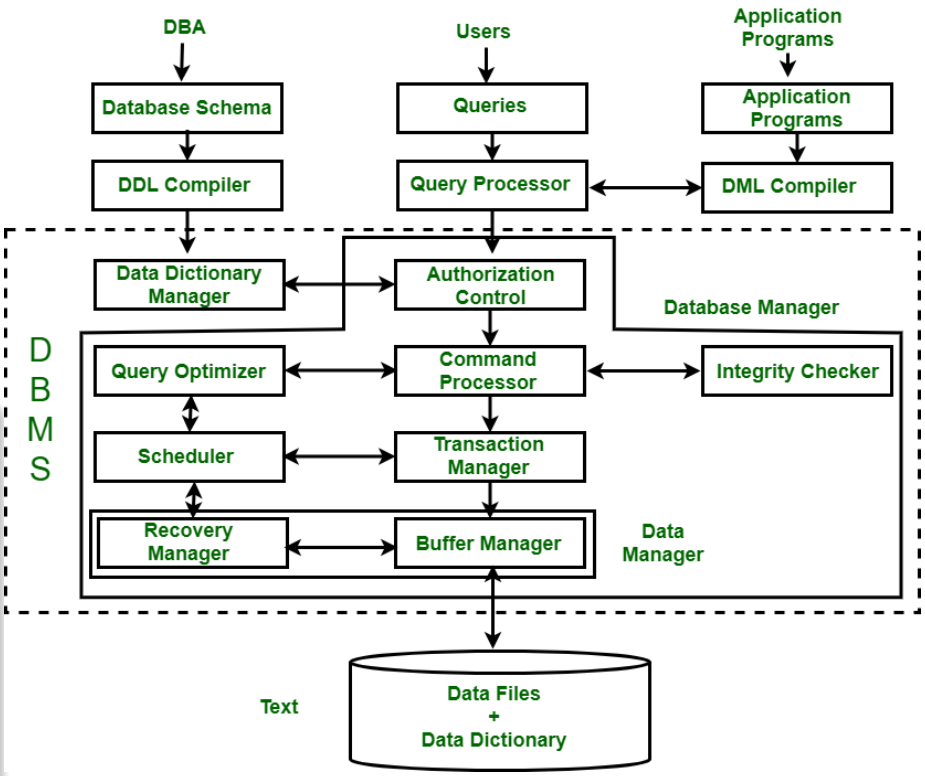
- Reliable & efficient storage.
- Concurrent access for many users.
- High-level languages to query data.

What is a DBMS? The "Librarian"

- **Database:** A large, integrated collection of data that models a real-world enterprise.
- **DBMS:** Software designed to store and manage databases.

The DBMS provides:

- Reliable & efficient storage.
- Concurrent access for many users.
- High-level languages to query data.



Why Not Just Use Files? (Bookstore Example)

Imagine building a bookstore with simple text files...

- Slow Searches: How to find a book by Victor Hugo? You must write a script to scan the entire 100GB file!
- Concurrency Issues: What if two users try to buy the last copy of a book at the same time?
- No Recovery: What if the power goes out after payment but before the book order is confirmed?
- Complex Logic: Every new query requires writing a new program.

A DBMS is a piece of software built to solve all of these hard problems for you.

Database Example

Create a database that models a digital music store to keep track of artists and albums.

Information we need to keep track of in our store:

- Information about Artists
- The Albums those Artists released

Flat File Strawman

Store our database as comma-separated value (CSV) files that we manage ourselves in application code.

- Use a separate file per entity.
- The application must parse the files each time they want to read/update records.

Artist(name, year, country)

"Wu-Tang Clan",1992,"USA"
"Notorious BIG",1992,"USA"
"GZA",1990,"USA"

Album(name, artist, year)

"Enter the Wu-Tang", "Wu-Tang Clan",1993
"St.Ides Mix Tape", "Wu-Tang Clan",1994
"Liquid Swords","GZA",1990

Flat File Strawman

Example: Get the year that GZA went solo.

Artist(name, year, country)
"Wu-Tang Clan",1992,"USA"
"Notorious BIG",1992,"USA"
"GZA",1990,"USA"

```
for line in file.readlines():
    record = parse(line)
    if record[0] == "GZA":
        print(int(record[1]))
```

Flat Files: Data Integrity

How do we ensure that the artist is the same for each album entry?

What if somebody overwrites the album year with an invalid string?

What if there are multiple artists on an album?

What happens if we delete an artist that has albums?

Flat Files: Implementation

How do you find a particular record?

What if we now want to create a new application that uses the same database? What if that application is running on a different machine?

What if two threads try to write to the same file at the same time?

Flat Files: Durability

What if the machine crashes while our program is updating a record?

What if we want to replicate the database on multiple machines for high availability?

Database Management System

A **database management system** (DBMS) is software that allows applications to store and analyze information in a database.

A general-purpose DBMS supports the definition, creation, querying, update, and administration of databases in accordance with some **data model**.

Data Models: A Blueprint for Data

What is a Data Model?

A collection of concepts for describing data. It's a blueprint that defines:

- **Structure:** How is the data organized? (e.g., tables, trees, graphs)
- **Operations:** What can you do with the data? (e.g., select, join, traverse)
- **Constraints:** What are the rules the data must follow? (e.g., student IDs must be unique)

Relational	Semi-Structured	Graph
(Tables)	(JSON/XML Trees)	(Nodes/Edges)

Data Models

Relational

- Relational← *most DBMSs*
- Key/Value Graph ← *Simple Apps / Caching*
- Graph ←*NoSQL*
- Document / JSON / XML / Object ← *NoSQL*
- *Wide-Column / Column-family ← NoSQL*
- *Array (Vector, Matrix, Tensor) ← ML*
- *Hierarchical*
- *Network*
- *Semantic*
- *Entity-Relationship*

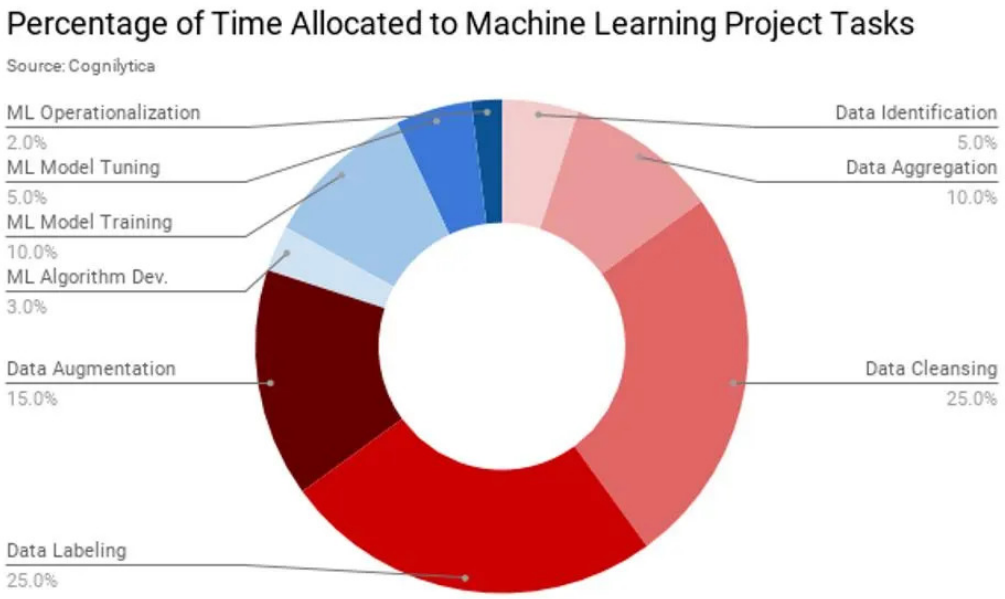
Why study relational databases?

Most important computer applications must manage, update and query datasets

- Bank, store, search app...

Data quality, quantity & timeliness becoming even more important with AI

- Machine learning = algorithms that generalize from data



The Relational Model: Core Concepts

- **Relation (Table):** An unordered set of tuples.
- **Tuple (Row):** A single record in a relation. It's formally a mapping from attribute names to values from their domains.
- **Attribute (Column):** A named column of a relation.
- **Domain:** The set of allowed values for an attribute (e.g., 'INTEGER', 'VARCHAR(20)').
- **Schema:** The structure of a relation, defined by its name and the set of its attributes. E.g., `Students(sid: INTEGER, name: VARCHAR(50), age: INTEGER)`

Key Principle: Atomicity

The value for any attribute in any tuple must be **atomic** (indivisible). You cannot have a list or a set as a value in a single cell. This is a cornerstone of First Normal Form (1NF).

The Relational Model: Keys

Keys are special sets of attributes that enforce uniqueness and link relations together.

Keys for Uniqueness

- **Superkey:** A set of attributes that uniquely identifies a tuple.
- **Candidate Key:** A *minimal* superkey (no attribute can be removed).
- **Primary Key:** The candidate key chosen by the database designer to be the main identifier for a relation. *Cannot be NULL.*

Keys for Relationships

- **Foreign Key:** A set of attributes in one relation that refers to the primary key of another relation. This creates a link and enforces **referential integrity**.

Database Case Study:building a book-selling website

Large data!
think about all books in the world or even in English
Where do we get started?

Q1: Who are the key people?

- At least two types:
  - Database admin (assuming they own all copies of all the books)
  - Customers who purchase books
  - Let's proceed with these two only
- Other people:
  - Sellers
  - Who deal with the warehouse of the books
  - ...

Q2: What should the user be able to do?

- i.e., what does the interface look like? (think about Amazon)
- Search for books
  - According to author, title, price range, ...
- Purchase books
- Add to wishlist
- ...

Q3: What should the platform do?

1. Returns books as searched by the authors
2. Check that the payment method is valid
3. Update no. of copies as books are sold
4. Manage total money it has
5. Add new books as they are published
6. ...

Q4: What are the desired and necessary properties of the platform?

- Should be able to handle a large amount of data
- Should be efficient and easy to use (e.g., search with authors as well as title)
- If there is a crash or loss of power, information should not be lost or inconsistent
  - Imagine a user was in the middle of a transaction when a crash happened, paid the money, but the book has not been purchased
- No surprises with multiple users logged in at the same time
  - Imagine one last copy of a book that two users are trying to purchase at the same time
- Easy to update and program
  - For the admin

That was the design phase (a basic one though)

Let's implement this!

How about:
Your favorite programming language
On data stored in large files

Image source: `https://www.flaticon.com/free-icon/girl_5986069`

Sounds simple!

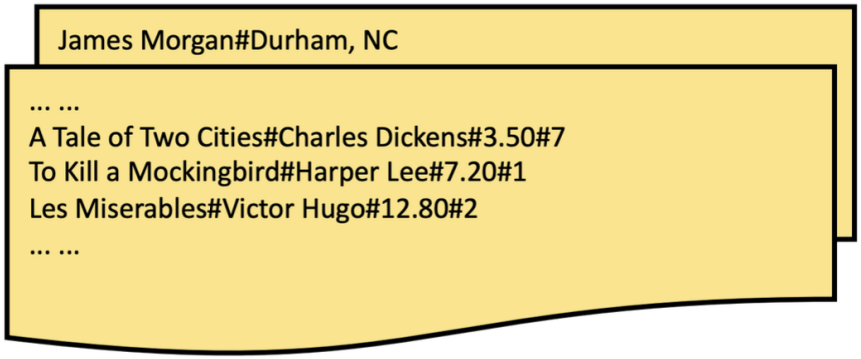
James Morgan#Durham, NC

... ...
A Tale of Two Cities#Charles Dickens#3.50#7
To Kill a Mockingbird#Harper Lee#7.20#1
Les Miserables#Victor Hugo#12.80#2
... ...

- Text files – for books, customers, ...
- Books listed with title, author, price and no. of copies
- Fields separated by #'s

Query by programming

James Morgan#Durham, NC



- James Morgan wants to buy "To Kill a Mockingbird"
- A simple script
  - Scan through the book files
  - Look for the line containing "To Kill a Mockingbird"
  - Check if there are more than 1 copy left
  - Charge James $7.20 and reduce the number of copies by 1
- Better idea than scanning?
  - Binary search (with file sorted on titles)
- What if he changes the "query" and wants to buy a book by Victor Hugo?

Revisit: What are the desired and necessary properties?

- Should be able to handle a large amount of data (Try to open a 10-100 GB file)
- Should be efficient and easy to use (e.g., search with authors as well as title) (Try to search both on a large flat file)
- If there is a crash or loss of power, information should not be lost or inconsistent
  Imagine a user was in the middle of a transaction when a crash happened, paid the money, but the book has not been purchased
- No surprises with multiple users logged in at the same time
  Imagine one last copy of a book that two users are trying to purchase at the same time
- Easy to update and program
  For the admin
  Imagine programmer's task
  Imagine adding a new book or updating copies (+ allow search) on a 10-100 GB text file

Solution?

DBMS = Database Management System

What is a DBMS?

A large, integrated collection of data

Models a real-world enterprise

- Entities e.g., Customers, Books
- Relationships (e.g., James purchases a tale of two cities)

A Database Management System (DBMS) is a piece of software designed to store and manage databases

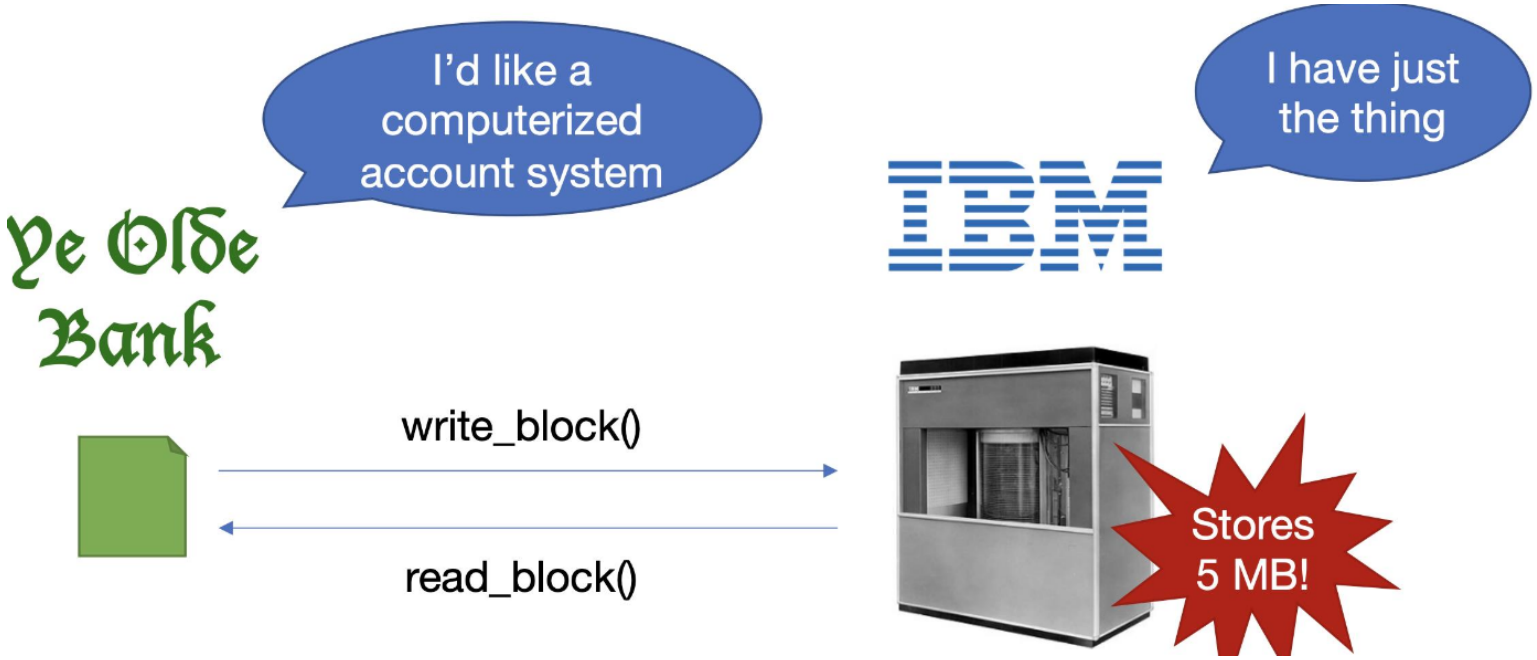
A DBMS takes care of all of the following (and more):

In an easy-to-use, efficient, and robust way

- **Optimization** to handle a large amount of data
- **Index** Should be efficient and easy to use (rather than such with authors as well as title)
- **Recovery** If the is loss of power, information should not be lost or inconsistent
- **Concurrency Control** No surprises with multiple users logged in at the same time
- **Declarative** Easy to update and program

Early Data Management

At first, each application did its own data management directly against storage (e.g., our book-selling website example)



Problems with App Storage Management

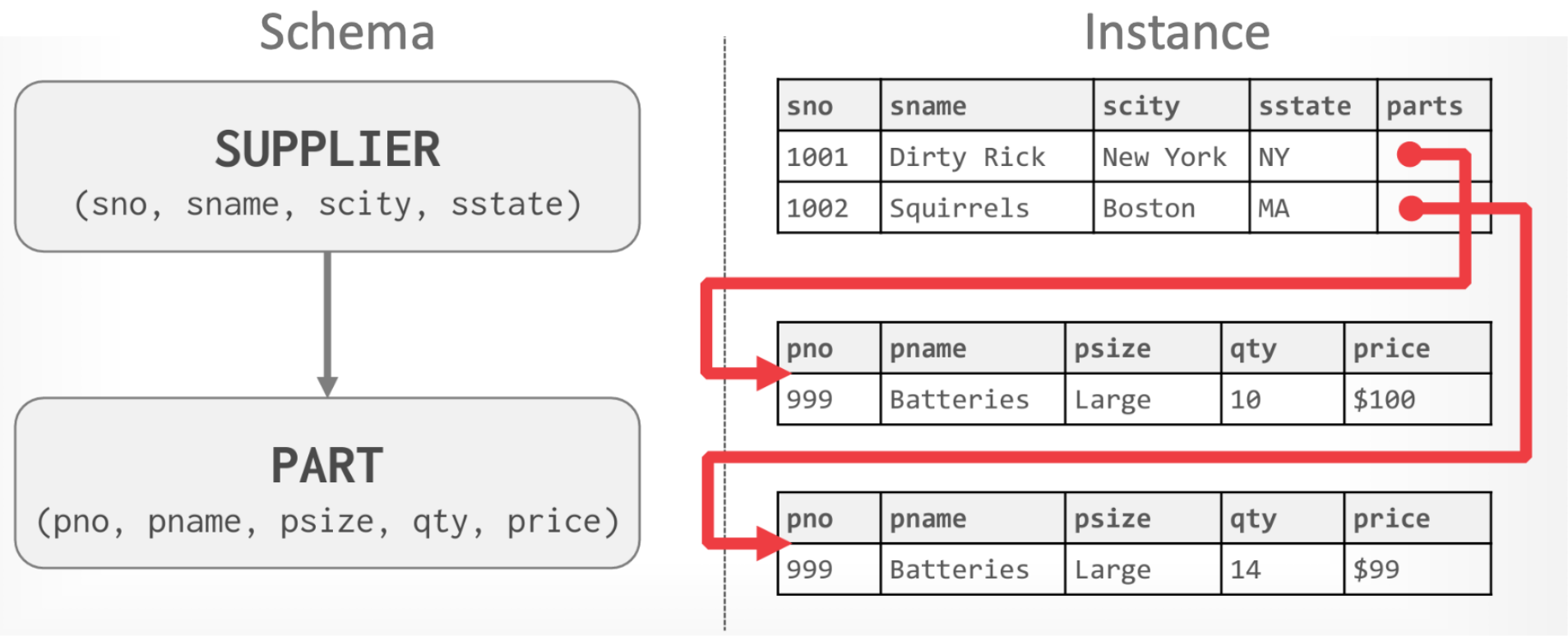
- How should we lay out and navigate data?
- How do we keep the application reliable?
- What if we want to share data across apps?

Every app is solving the *same* problems!

1960s – IBM IMS

- Information Management System
- Early database system developed to keep track of purchase orders for Apollo moon mission.
  - Hierarchical data model.
  - Programmer-defined physical storage format.
  - Tuple-at-a-time queries.

Hierarchical Data Model



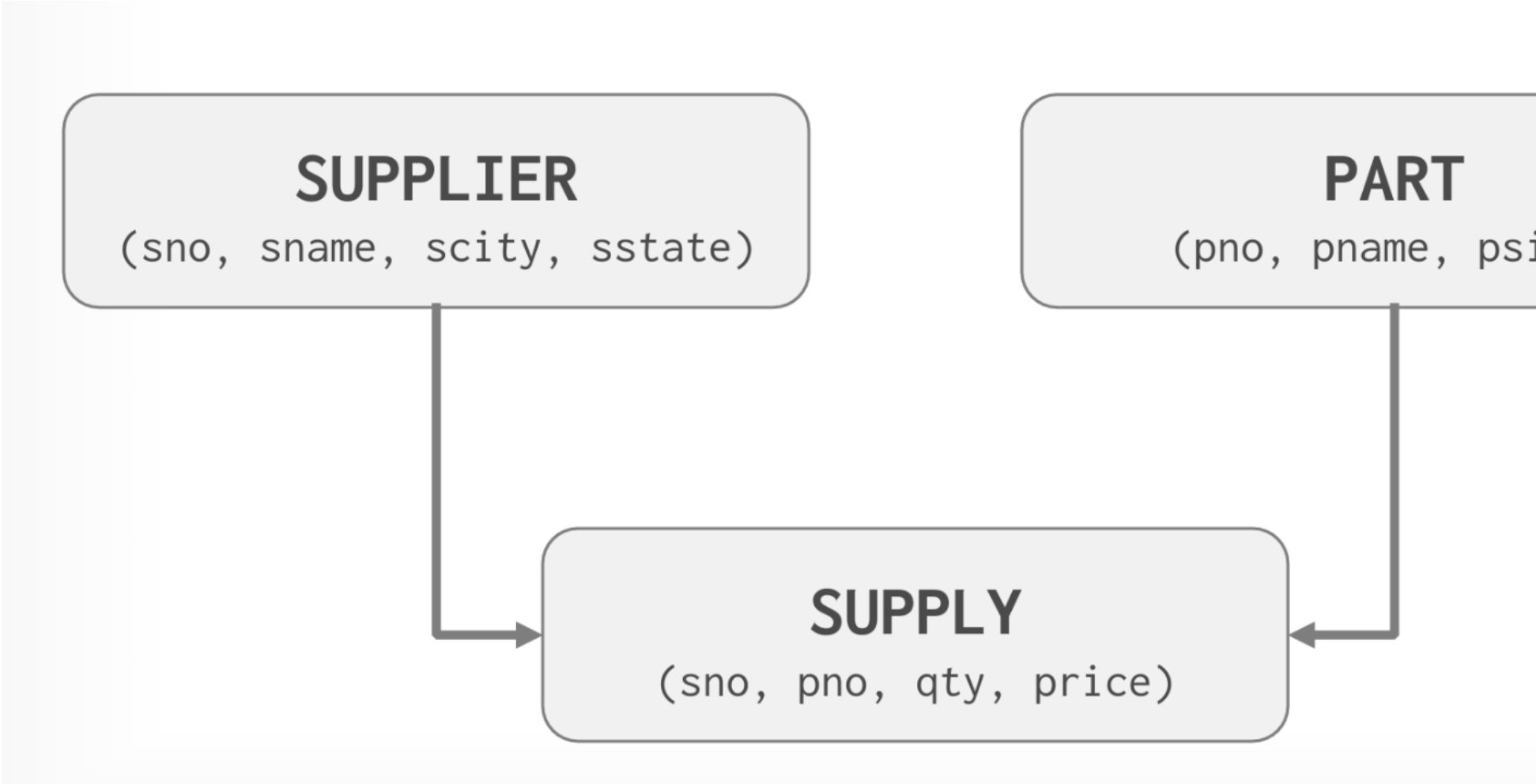
1970s - Relational data model

Ted Codd was a mathematician working at IBM Research. He saw developers spending their time rewriting IMS programs every time the database's schema or layout changed.

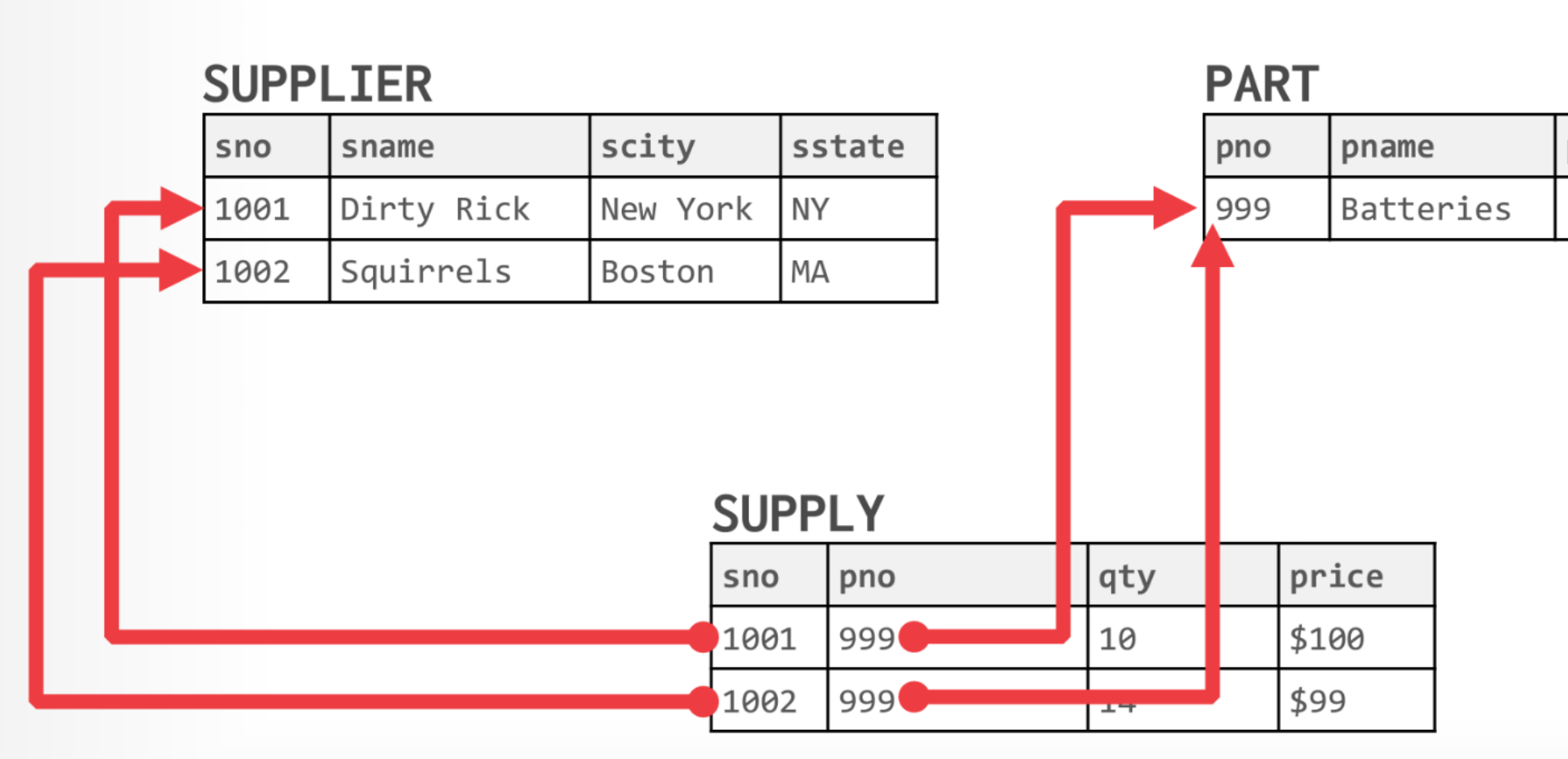
Database abstraction to avoid this maintenance:

- Store database in simple data structures
- Access data through set-at-a-time high-level language.
- Physical storage left up to implementation.

Relational Data Model - schema



Relational Data Model - instance



SUPPLIER

sno	sname	scity	sstate
1001	Dirty Rick	New York	NY
1002	Squirrels	Boston	MA

PART

pno	pname
999	Batteries

SUPPLY

sno	pno	qty	price
1001	999	10	$100
1002	999	14	$99

Data independence

Concept: Applications do not need to worry about *how the data is structured and stored*

Logical data independence: protection from changes in the *logical structure of the data*

- I.e. should not need to ask: can we add a new table, or remove a field in a table without rewriting the application?

Physical data independence: protection from *physical layout changes*

- I.e. should not need to ask: which disks are the data stored on? Is the data indexed?

One of the most important reasons to use a DBMS

Data model

A notation for describing data or information

Consists of:

- Structure of the data
- Operations on the data
- Constraints on the data

Structure of the data

- Referred to as a "conceptual model" of the data
- Higher level than "physical models" or data structures like arrays and lists
- Example: a relation consists of a schema, attributes, and tuples

title	year	length	genre
Oldboy	2003	120	mystery
Ponyo	2008	103	anime
Frozen	2013	102	anime

Operations on the data

Usually a limited set of operations that can be performed

- Queries (operations that retrieve information)
- Modifications (operations that change the database)
- Relation algebra

This is a strength, not a weakness

- Programmers can describe operations at a very high level
- The DBMS implements them efficiently
- Not easy to do when coding in C

```
SELECT *
FROM Movies
WHERE studioName = 'Disney'
AND year = 2025;
```

The relational model

Constraints

- E.g., Genre must be one of a fixed list of values, no two movies can have the same title

title	year	length	genre
Oldboy	2003	120	mystery
Ponyo	2008	103	anime
Frozen	2013	102	anime

The semi–structured model

Structure:

- Resembles tree/graph;
- Represent data by hierarchically nested tagged elements.

Operations:

- Follow paths from element to subelements.

Constraints:

- Types associated with tags (e.g., <Length> is integer; each <Movie> has a <Length>).

```
<Movies>
 <Movie title="Oldboy">
  <Year>2003</Year>
  <Length>120</Length>
  <Genre>mystery</Genre>
 </Movie>
 <Movie title="Ponyo">
  <Year>2008</Year>
  ...
</Movies>
```

The key-value model

Structure

- (key, value) pairs
- Key is a string or integer
- Value can be any blob of data

Operations

- get (key), put(key, value)
- Operations on values not supported

Constraints

- E.g., key is unique, value is not NULL

key	value
1000	(oldboy, 2003)
1001	(ponyo, 2008)
1002	(frozen, 2013)

Comparison of modeling approaches

Relational model

- Simple/limited but reasonably versatile;
- Limited, but useful operations
- Efficient access to large data
- A few lines of SQL can replace 1000s of lines of C code
- Preferred in DBMSs

Semi–structured model

- more flexible, slower to query.

Key–value model

- even more flexible, cannot query.

Basics of the relational model

- A database is a collection of **relations** (tables)
- Each relation has a set of **attributes** (columns),
- Each attribute has a name and **domain** (type)
  - Set–valued attributes are not allowed (e.g., you cannot store a list/set of bars in a cell, all cells contain atomic values)
- Each relation contains a set of tuples (rows)
  - Each tuple has a value for each attribute of relation
  - Ordering of rows doesn't matter (output is in some order)

Basics of the relational model

- Relation: two-dimensional table containing data
- Schema: relation name and set of attributes
  - Movies(title, year, length, genre)
- Database schema: set of schemas for the relations of a database
- A tuple has one component for each attribute
  - (Oldboy, 2003, 120, mystery)

title	year	length	genre
Oldboy	2003	120	mystery
Ponyo	2008	103	anime
Frozen	2013	102	anime

Equivalent representations of a relation

A relation is a set of tuples (not a list). A schema is a set of attributes (not a list). Hence, tuple/attribute order is immaterial.

title	year	length	genre
Oldboy	2003	120	mystery
Ponyo	2008	103	anime
Frozen	2013	102	anime

$\Leftrightarrow$

year	genre	title	length
2013	anime	Frozen	102
2003	mystery	Oldboy	120
2008	anime	Ponyo	103